# LLD Notes — Phase 4

Engineering concerns around the design: requirements · entities · UML · layers · repositories · testing · exceptions · concurrency · thinking out loud | Self-study, answers at the back

> **Colour key. Purple = the interviewer's words · Blue = a note or an exercise · Red = a common mistake · Green = a rule to remember · Amber = how to say it in an interview.**

### Contents


**1. Requirements** — functional vs non-functional · the first five minutes · exercise 1

**2. Entities, actors, relationships** — from a paragraph to a list of classes · exercise 2

**3. UML class diagrams** — the six arrows · multiplicity · what to draw in five minutes · exercise 3

**4. Layering** — entity / service / repository / driver · where each rule goes · exercise 4

**5. Repositories** — in-memory storage behind an interface · exercise 5

**6. Dependency injection and testing** — "how would you test this?" · fakes · exercise 6

**7. Exceptions and errors** — checked vs unchecked · custom exceptions · throw or return a result · exercise 7

**8. Concurrency** — lost updates · check-then-act · the toolbox · what to lock · deadlock · optimistic vs pessimistic · several servers · exercise 8

**9. Thinking out loud** — a timeline for the round · what to say · exercise 9

**10. Everything together** — a car rental service, built end to end, with tests

**Answers** to exercises 1–9 · **cheat sheet** · mistake log


## How to use this PDF

Phase 3 taught the patterns. Phase 5 used them on 24 problems. This phase covers what sits *around* the design: the questions you ask first, how you draw it, how the code is split into layers, how it's tested, what happens when things fail, and what happens when two people click at the same moment. These are the follow-up questions that separate a passing answer from a strong one.

Every section has the same shape: the idea with one example, the rules, the common mistakes, a line to say in the interview, then an exercise. Do the exercise on paper before reading the answer at the back. Section 8 (concurrency) is the longest, because it's the most common follow-up and the most often under-prepared.

> **One example runs through every section:** a car rental service in Pune, like Zoomcar. Each section adds one piece, and section 10 shows the whole thing as working code with tests. All code in this PDF was compiled and run; where a result changes from run to run (races), the numbers shown are from real runs.

# 1. Requirements: functional vs non-functional

> **The interviewer says:** "Design a car rental system, like Zoomcar."

Two candidates get this prompt. **Rohit** starts typing `class Car` at once. Twenty minutes later the interviewer asks, "Can someone rent for three hours?" His design counts days everywhere, and he spends the rest of the round patching it. **Sneha** spends four minutes asking questions, writes the answers down as a numbered list, and builds exactly that. She finishes with time to talk about concurrency. Same skill at coding; very different result.

## Two kinds of requirement

| Kind | What it describes | Car rental examples |
|---|---|---|
| **Functional** | *What* the system does: actions users take and what they get back. Usually verbs. | Search cars by city and dates · book a car · cancel a booking · price a booking · pick up and return |
| **Non-functional** | *How well*, or under what conditions: qualities the whole system must have. | Never book one car twice for the same dates, even when two people click together · pricing rules will change often · storage is in memory now, a database later · the booking logic must be testable without a database |

Functional requirements decide your **methods**. Non-functional ones decide your **structure**: where the interfaces go, what gets locked, what's passed in. That's why interviewers care so much about them, and why "it should be scalable" is not a useful non-functional requirement: it doesn't change anything you'd draw.

## The questions to ask, in groups

| Group | Questions (car rental) |
|---|---|
| **Actors** | Who uses it? Customers only, or also branch staff and admins? Any outside systems (payment, maps)? |
| **Core flows** | What are the three most important things a user does? (Search, book, cancel.) Is pick-up and return in scope? |
| **Rules and numbers** | Rent by day or by hour? How is the price worked out? Can a car be picked up the same day the last renter returns it? What's the refund rule? |
| **Edge cases** | What if two people book the same car at once? A booking in the past? A car that doesn't exist? Cancel twice? |
| **Non-functional** | Many users at once? Which rules will change? In-memory storage, or a database? Several servers? |
| **Scope cuts** | Can I skip payment details, login and the UI? One city per car? |

## Write them down as numbered lines

Turn every answer into a line you can point at later. This is the format every Phase 5 problem used:

| | |
|---|---|
| **R1** | Cars have a registration number, a model, a type (HATCHBACK Rs 1500/day, SEDAN Rs 2200/day, SUV Rs 3000/day) and a city. |
| **R2** | A customer searches a city for cars free for a whole date range. Rentals are by day; the return day is not charged, so the next renter can pick up that day. |
| **R3** | Booking charges the customer up front. Refused if the car doesn't exist, the pickup date is in the past, the return date isn't after the pickup date, the car is taken for any of those days, or payment fails. |
| **R4** | A booking can be cancelled once; the car becomes free again. |
| **NF1** | Two customers booking the same car at the same moment: only one succeeds. Bookings for different cars don't wait for each other. |
| **NF2** | Pricing rules will change (weekend rates, long-rental discounts). |
| **NF3** | Storage is in memory for now; it will move to MySQL. The booking logic must be unit-testable without a database or a real payment provider. |

**Out of scope:** login, payment details, hourly rentals, late-return fines, the UI.

## How each non-functional requirement shows up in the design

| When you hear… | …your design gets |
|---|---|
| "These rules will change" (pricing, matching, refunds) | An interface with one class per rule: **Strategy** |
| "Many users at the same time" | Find the check-then-act step; lock the smallest thing that covers it (section 8) |
| "Notify / log / update X when Y happens" | **Observer**: listeners, so the core flow doesn't know who reacts |
| "In memory now, a database later" | A **repository interface** with an in-memory class behind it (section 5) |
| "Must be testable" / anything with time or randomness | Everything passed in through the constructor, including a `Clock` (section 6) |
| "Several servers" | Say how: database constraints or conditional updates, not `synchronized` (section 8.8) |
| "Every change must be traceable" | An append-only log of events next to the current state |

> **Machine coding vs discussion.** In a machine-coding round (Flipkart, Swiggy, Razorpay), the requirements usually come written down. Still ask two or three questions, and write your assumptions at the top of your code in a comment. In a discussion round, nobody hands you a list: driving these five minutes *is* part of what's graded.

> **Rules.** Spend three to five minutes, not fifteen: six to eight questions, grouped. When the interviewer says "you decide", state an assumption out loud and move on. Turn every answer into an R or NF line, and list what's out of scope. Every NF line should change something you'd draw; if it doesn't, it's not worth writing.

**Common mistakes**
- Starting to code before knowing whether it's days or hours, one city or many.
- Twenty questions, one at a time, with no direction: it reads as stalling.
- Asking about a million users in a 90-minute machine-coding round, where it changes nothing.
- "It should be fast and scalable", with no design decision attached.
- Never raising concurrency, when two people booking the same thing is the obvious follow-up.

**Say it like this.** "Before I design, let me take three or four minutes on scope. Who uses it: just customers, or staff too? The main flows are search, book and cancel; is pick-up and return in scope? Is renting by day or by hour? ... I'll assume day-based rentals and one city per car, and leave payment details and login out. Two things I'll design for: two people booking the same car at once, and pricing rules changing."

**Exercise 1** (15 min). "*Design a membership and class booking app for a gym chain in Pune.*" *Payments are out of scope.*
**Part A.** Write six questions you'd ask, grouped as above.
**Part B.** Mark each line F (functional), NF (non-functional) or OUT (out of scope):
1. Members can book a spot in a Zumba class. · 2. Two members must never get the last spot at the same moment. · 3. The app must take UPI payments. · 4. A trainer can see who booked her class. · 5. New plans (student, couple, corporate) will be added often. · 6. Members get a reminder an hour before class. · 7. Booking must respond within 200 ms. · 8. A member can cancel up to 2 hours before class. · 9. Storage is in memory now but will move to MySQL. · 10. The class page shows how many spots are left. · 11. Every booking and cancellation must be kept for disputes. · 12. Booking logic must be testable without a database.
**Part C.** For each NF line, write in one line what it changes in the design.

## 2. Entities, actors and relationships

You have the requirements. Now turn a paragraph into a list of classes. The old trick still works as a *starting point*: **nouns tend to become classes, verbs tend to become methods**. Then you filter.

### Step 1: underline the nouns and verbs

**The requirements, as a paragraph:** "A customer *searches* a city for cars free for a date range. Each car has a registration number, a model, a colour and a type: hatchback, sedan or SUV, each with its own price per day. The customer *books* a car; the booking is *paid* through a payment gateway and the customer gets an invoice. A vehicle can't have two reservations for the same day. A customer can *cancel* a booking. Admins *add* cars."

### Step 2: decide what each noun is

| Noun | It's a... | Why |
|---|---|---|
| car, booking | **Entity** (a class with an ID) | Each one is a particular thing that changes over time. Two bookings with the same dates are still two bookings. |
| registration number, model, colour, price per day | **Attribute** (a field) | A single value describing something else. A `Colour` class would hold one string and do nothing. |
| date range | **Value object** (a record) | Two values that only make sense together, with behaviour of their own ( `overlaps` , `days` ). No ID: two ranges with the same dates *are* the same range. |
| type (hatchback, sedan, SUV) | **Enum** | A small fixed set that differs only in data (the price). Three subclasses would differ in nothing but a number. |
| customer, admin | **Actors** | People who use the system. Often just a name or an ID in an LLD round, unless they have their own rules. |
| payment gateway | **Actor (a system)** → an interface | Outside our code. We call it through an interface so we can swap providers and fake it in tests. |
| vehicle, reservation | **Synonyms** | Same as car and booking. Pick one name and use it everywhere. |
| city | Attribute (for now) | A string on the car is enough until cities get their own data (branches, taxes). |
| invoice | Out of scope | Mentioned, but nobody asked us to build it. Say so. |

**Entity or value object? One test.** "If two of them have exactly the same fields, are they the same thing?" Two Rs 500 notes: yes, so a value. Two customers both named Asha Kulkarni: no, so an entity with an ID. Value objects are records: no ID, never change, compared by their fields.

### Step 3: verbs become methods, on the class that has the data

| Verb | Where it goes | Why there |
|---|---|---|
| cancel a booking | `Booking.cancel()` | It only needs the booking's own status. Put the method where the data is. |
| do two date ranges clash? | `DateRange.overlaps()` | Only needs the two ranges. |
| is this car free for these dates? | The service, using the repository | Needs *all* bookings of the car, which no single booking knows. |
| price of a booking | A `PricingStrategy` | A rule that will change (NF2), so it gets its own interface. |
| book a car | `RentalService.book()` | Touches the car, the bookings, the price and the payment: a use case, so a service method. |
| search, add a car | Service, then repository | Finding and storing is the repository's job (section 5). |

### Step 4: relationships — has-a or is-a?

Most relationships are **has-a**: a booking has a car ID and a date range. Use **is-a** (inheritance) only when the subtypes *behave* differently and an object never changes type. Hatchback, sedan and SUV behave the same and differ only in a number, so they're an enum. Roles that can change (a customer who becomes an admin) are a field or a set of roles, never a subclass: an object can't change its class.

**Rules.** Nouns and verbs are where you start, not the answer. Differ by data → one class with fields, or an enum. Put each method on the class that has the data it needs; a rule that spans several objects goes in a service. Anything outside your code (payment, SMS, maps) becomes an interface.

**Common mistakes**

- A class for every noun: `Colour`, `Model`, `RegistrationNumber`, each holding one string.
- One giant `RentalSystem` class with every method in it.
- `Admin extends Customer extends Person` for roles that change.
- Entities that are only getters and setters, with every rule in a `Manager` class.
- Forgetting the actors that are systems: the payment gateway, the SMS provider.

**Say it like this.** "From the requirements, my entities are Car and Booking. Colour and model are just fields. DateRange is a small value object, because overlap checks belong with the dates. Car type is an enum, since the types only differ in price. The payment gateway is outside our system, so it's an interface."

**Exercise 2** (15 min). "*A gym chain has branches. Each branch runs classes (Zumba, yoga, spinning) with a trainer, a start time and a number of spots. A member has a plan (monthly, quarterly, student) and a membership end date. Members book spots in classes and can cancel. Plans differ in price and in how many classes a week they allow. Trainers see their class lists. Members get an SMS reminder before class.*"
**Part A.** For each noun, say: entity, attribute, value object, enum, actor, or out of scope.
**Part B.** Where does each behaviour go? (a) does this class have a free spot? (b) the price of a plan for a student; (c) cancel a booking; (d) is the membership active on a given date? (e) send the reminder; (f) book a member into a class.

## 3. UML class diagrams and multiplicity

You'll be asked to draw, on a whiteboard or in Excalidraw, in about five minutes. Nobody wants perfect UML; they want boxes, the right arrows and numbers at the ends, so they can see your design at a glance.

### A class box

```
┌──────────────────────────┐
│ Booking                  │   name (italics or «abstract» if abstract,
├──────────────────────────┤         «interface» above the name for an interface)
│ - id: String             │   fields: name: type    (- private, + public)
│ - range: DateRange       │
│ - status: BookingStatus  │   only the fields that matter for the design
├──────────────────────────┤
│ + cancel(): void         │   methods: name(params): return type
│ + isActive(): boolean    │   skip getters, setters and constructors
└──────────────────────────┘
```

### The six relationships

This PDF (and Phase 5) draws them with text characters, which is also how you'd type them in a chat or a doc:

| Drawn as | Name | In Java | Example |
|---|---|---|---|
| `A ——> B` | Association | A has a field of type B, and uses it over time | `Booking ——> Car` |
| `A ◇—— B` | Aggregation | A holds Bs, but Bs can live without A or move to another A | `Branch ◇—— Car` (a car can move branches) |
| `A ◆—— B` | Composition | Bs are created by A and die with A; a B belongs to one A only | `Order ◆—— OrderLine` |
| `B ——▷ A` | Inheritance | `class B extends A`; the triangle points at the parent | `SavingsAccount ——▷ Account` |
| `B - - -▷ A` | Realization | `class B implements A` | `PerDayPricing - - -▷ «interface» PricingStrategy` |
| `A - - -> B` | Dependency | A uses B briefly: a parameter or a local variable, not a field | `RentalService - - -> DateRange` |

In Phase 5 diagrams, `Service ——> «interface» X ◁- - - Impl` means: the service holds an X, and Impl implements X. The triangle always sits next to the thing being extended or implemented.

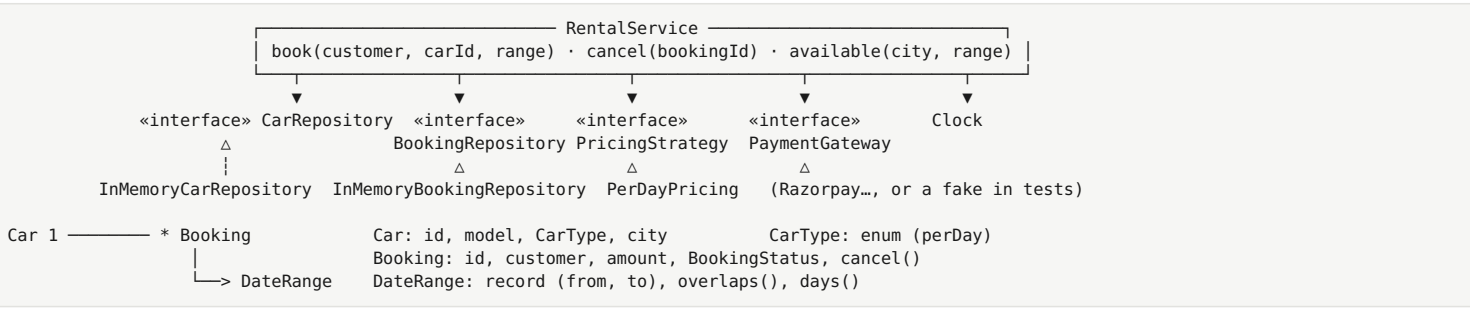
### Multiplicity: how many at each end

| Mark | Means | Read it as |
|---|---|---|
| `1` | exactly one | `Car 1 —— * Booking`: "a car has any number of bookings; each booking is for exactly one car". The number sits at the far end from the class it describes: the `*` next to Booking says how many bookings one car has. |
| `0..1` | none or one | |
| `*` or `0..*` | any number, including none | |
| `1..*` | at least one | |

**Composition or aggregation? Two questions.** (1) If the whole is deleted, do the parts die too? (2) Can a part belong to two wholes, or move to another? Die with it and never shared: composition (filled diamond). Otherwise: aggregation, or just a plain association. If you're not sure, use a plain arrow: a wrong diamond costs more than no diamond.

### What to draw in five minutes, in this order

(1) The entities, each with two to four key fields. (2) Arrows between them, with multiplicity. (3) An interface for each thing that varies, with its implementation hanging off it. (4) The service in the middle, holding the interfaces. (5) The repositories. Here is the car rental, drawn that way:




### When they ask "walk me through a booking": a sequence

Sometimes the interviewer wants the flow, not the classes. A numbered list of calls is enough:

```
Driver          -> RentalService.book("Asha", "MH12-2002", 24-27 Dec)
RentalService   -> Clock: is the pickup date in the past?              no
RentalService   -> CarRepository.findById("MH12-2002")                 the City sedan
RentalService   :  lock this car
RentalService   -> BookingRepository.findByCar("MH12-2002")            check for overlaps: none
RentalService   -> PricingStrategy.price(car, range)                   Rs 6600
RentalService   -> PaymentGateway.charge("Asha", 6600)                 ok
RentalService   -> BookingRepository.save(B1)
RentalService   :  unlock; return B1 to the driver
```

**Common mistakes**

- The inheritance triangle pointing at the child instead of the parent.
- Filled diamonds everywhere, including for things that are shared or outlive the whole.
- No multiplicity, so nobody can tell one-to-one from one-to-many.
- Twenty-five boxes with every getter and setter; the design disappears in the detail.
- Arrows in both directions "just in case": every arrow is a dependency someone has to maintain.

**Say it like this.** "I'll draw the entities first, then the things that vary as interfaces. A car has many bookings, each for one car. The service sits in the middle and only knows interfaces: the repositories, pricing and the payment gateway. Should I walk you through the booking flow as a sequence?"

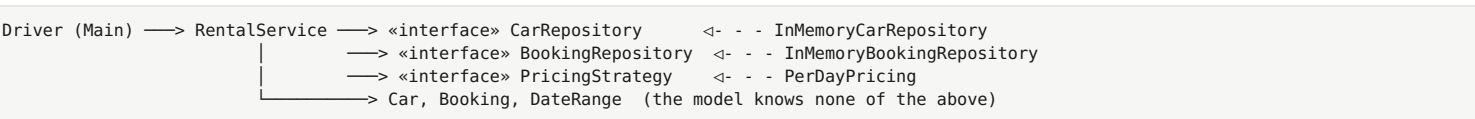
**Exercise 3** (15 min).
**Part A.** Write the notation (arrow and multiplicity) for: (1) a branch runs many classes, and a class is at one branch; (2) `StudentPlan` implements `Plan`; (3) a class booking can't exist without its class, and is deleted with it; (4) a trainer teaches many classes, and can move to another branch; (5) `BookingService` takes a `LocalDate` as a parameter only; (6) `PremiumMember extends Member`.
**Part B.** Draw the class diagram for the gym from exercise 2: entities, enums, value objects, interfaces, the service and repositories, with multiplicity.

## 4. Layering: entity, service, repository

Think of a bakery on FC Road. The **counter staff** take orders and hand over boxes, but never bake. The **baker** follows recipes, but never talks to customers. The **store room** keeps flour and butter, but doesn't decide what to bake. The **cakes** know their own weight and flavour. Each one talks only to the next, so a new counter (a phone order line) doesn't change the baking.

### The four layers

| Layer | Its job | Knows about | In Spring |
|---|---|---|---|
| **Driver / controller** | Read input, call one service method, format the output. No business rules. | The service | `@RestController` |
| **Service** | One method per use case (book, cancel). Rules that need several objects; locks and transactions. | Repositories (as interfaces), strategies, entities | `@Service` |
| **Repository** | Store and find. No business rules. | Entities | `@Repository`, `JpaRepository` |
| **Entity / model** | Data, plus the rules about itself (`Booking.cancel()` checks its own status). | Nothing above it | `@Entity` |




Arrows point one way: down and inward. The model never imports a repository; the service never prints; the driver never decides a price.

### Where does each rule go?

| The rule… | Goes in | Car rental example |
|---|---|---|
| is about one object's own state | the **entity** | a cancelled booking can't be cancelled again |
| needs several objects, or a search | the **service** | no two active bookings of one car overlap |
| will change, or has several versions | a **strategy**, called by the service | the price of a booking |
| is about input or output format | the **driver** | dates come as `yyyy-mm-dd`; errors print their message |
| is about storing or finding | the **repository** | find all bookings of a car |

### Packages, for a machine-coding round

```
com.rental
├── model          Car, CarType, Booking, BookingStatus, DateRange
├── repository     CarRepository, BookingRepository, InMemoryCarRepository, InMemoryBookingRepository
├── service        RentalService, PricingStrategy, PerDayPricing, PaymentGateway
├── exception      RentalException, CarNotFoundException, CarNotAvailableException, PaymentFailedException
└── Main           the driver: reads commands, wires everything, prints results
```

If the round wants a single file (like the Phase 5 solutions), keep the same order with a comment line between groups, as section 10 does.

**Entities aren't just data bags.** If every entity has only getters and setters, every rule ends up in services, and the same check gets copied into three places. A rule about one object's own state belongs on that object. Services coordinate; entities protect themselves.

**Common mistakes**

- Everything in `Main`: parsing, rules, storage and printing in one method.
- A service that calls `System.out.println`: it can't be reused behind a REST API or in a test.
- An entity that calls a repository (`booking.save()`), so the model depends on storage.
- A repository with rules in it (`findAndBookIfFree`).
- A `Utils` class that collects everything nobody wanted to place.

**Say it like this.** "Main only parses and prints. The service has one method per use case and holds the rules that span several objects. Repositories just store and find. Entities guard their own state, like a booking refusing a second cancel. It's the same split as Spring's controller, service and repository."

**Exercise 4** (10 min). Which layer should each numbered part be in, and what's wrong with it here?

```
class BookingManager {
    static Map<String, FitnessClass> classes = new HashMap<>();       // (1)

    void handle(String line) {
        String[] p = line.split(" ");                                  // (2)
        FitnessClass c = classes.get(p[1]);
        if (c.booked.size() >= c.capacity) {                           // (3)
            System.out.println("Class full");                          // (4)
            return;
        }
        int price = p[2].equals("STUDENT") ? 150 : 250;                // (5)
        c.booked.add(p[0]);                                            // (6)
        System.out.println(p[0] + " booked, Rs " + price);             // (7)
    }
}
```

## 5. Repositories: in-memory storage behind an interface

Every LLD problem needs to store things, and no interviewer wants a database in a 60-minute round. The expected shape is a small **interface in the language of the domain**, with a `Map` behind it. The service only ever sees the interface.

## The shape

```
interface CarRepository {
    void save(Car car);
    Optional<Car> findById(String id);
    List<Car> findByCity(String city);
}

interface BookingRepository {
    void save(Booking booking);
    Optional<Booking> findById(String id);
    List<Booking> findByCar(String carId);
}

class InMemoryCarRepository implements CarRepository {
    private final Map<String, Car> cars = new ConcurrentHashMap<>();

    public void save(Car car) { cars.put(car.id(), car); }
    public Optional<Car> findById(String id) { return Optional.ofNullable(cars.get(id)); }

    public List<Car> findByCity(String city) {
        return cars.values().stream().filter(c -> c.city().equals(city))
                .sorted(Comparator.comparing(Car::id)).toList();
    }
}

class InMemoryBookingRepository implements BookingRepository {
    private final Map<String, Booking> byId = new ConcurrentHashMap<>();
    private final Map<String, List<Booking>> byCar = new ConcurrentHashMap<>();    // an index

    public void save(Booking b) {
        if (byId.put(b.id, b) == null)                          // new, not an update
            byCar.computeIfAbsent(b.carId, k -> new CopyOnWriteArrayList<>()).add(b);
    }

    public Optional<Booking> findById(String id) { return Optional.ofNullable(byId.get(id)); }

    public List<Booking> findByCar(String carId) {
        return List.copyOf(byCar.getOrDefault(carId, List.of()));    // a copy, not our list
    }
}
```

## Why the interface is worth four lines

| Reason | What it gives you |
|---|---|
| Storage will change (NF3) | MySQL later is a new class, `JdbcBookingRepository`. The service doesn't change. |
| Testing | Tests use the in-memory class: fast, no setup, nothing to clean up. |
| It's what Spring does | `interface BookingRepository extends JpaRepository<Booking, String> { List<Booking> findByCarId(String carId); }` and Spring writes the class for you. Your in-memory version is the same contract, written by hand. |

## Details interviewers notice

| Detail | Do this |
|---|---|
| Not found | Return `Optional`, never `null`. The caller decides: `orElseThrow(() -> new CarNotFoundException(id))`. |
| Returning lists | Return a copy (`List.copyOf`) or an unmodifiable view, never your internal map or list. Otherwise a caller can change your storage behind your back. |
| Threads | `ConcurrentHashMap` keeps the map itself safe. It does **not** make the service's "check, then save" safe; that's section 8. |
| IDs | One place creates them: an `AtomicInteger` in the service (as here), or a `nextId()` on the repository. |
| Frequent queries | Add an index map (`byCar` above), and keep it in step inside `save`. That's a trade-off: faster reads, a bit more to keep correct. |
| After a change | Call `save()` again. With a map, the object you changed is the stored one, so it seems unnecessary; a database-backed repository hands you a copy, and only `save()` writes it back. Calling it makes your service work with both. |

**Rules.** One repository per entity, named in domain words (`findByCar`, not `query3`). Store and find only: no business rules. `Optional` for one result, a copied list for many. Keep the interface small: add a query when a use case needs it, not "in case".

**Common mistakes**

- `public static Map` used as a global database from everywhere.
- `findById` returning `null`, so a `NullPointerException` appears three calls later.
- Returning the internal map, so callers can add or remove entries directly.
- Rules inside the repository: `renewIfExpired`, `bookIfFree`.
- Keying the map by something that isn't unique, like a name.

**Say it like this.** "I won't use a database: each entity gets a repository interface with a ConcurrentHashMap behind it. The service only knows the interface, so switching to MySQL, or to Spring Data JPA, is a new class, and tests use the in-memory one."

**Exercise 5** (10 min).
**Part A.** What's wrong with each, and what's the fix?

```
(a) public static Map<String, Member> MEMBERS = new HashMap<>();
(b) Member findById(String id) { return members.get(id); }
(c) Map<String, Member> findAll() { return members; }
(d) void save(Member m) { members.put(m.name, m); }
(e) boolean renewIfExpired(String id, LocalDate today) { ... }     // in MemberRepository
```

**Part B.** Write just the method signatures of a `ClassBookingRepository` for the gym that supports: book (store), cancel (find by ID, store again), "who booked this class?" and "what has this member booked this week?".

# 6. Dependency injection and testability

**The interviewer asks:** "How would you unit test this?"

It sounds like a testing question. It's really a **design** question: can you run your service without a real database, a real payment provider and today's date? If you can't, your classes are too tightly tied to the things around them.

Picture a tiffin service in Kothrud whose recipe says "buy vegetables from Shinde's stall on Paud Road". When Shinde's stall is shut, there are no tiffins. To try a new recipe, you have to go to Paud Road. If the recipe said "vegetables from any supplier", any stall would do, including a basket in your own kitchen for testing.

## Hidden dependencies: what makes code hard to test

| Inside a service… | …means a test can't avoid |
|---|---|
| `new RazorpayGateway()`, `new SmsSender()` | real payments, real SMS |
| `Database.BOOKINGS` (a static map), singletons | shared state leaking between tests |
| `LocalDate.now()`, `System.currentTimeMillis()` | today's date: "is this in the past?" gives a different answer tomorrow |
| `new Random()`, `UUID.randomUUID()` used for decisions | a different result every run |

## The fix: constructor injection

Everything the service needs comes in through its constructor. Nothing is created inside.

```
class RentalService {
    private final CarRepository cars;
    private final BookingRepository bookings;
    private final PricingStrategy pricing;
    private final PaymentGateway payments;
    private final Clock clock;
    private final Map<String, Object> carLocks = new ConcurrentHashMap<>();
    private final AtomicInteger count = new AtomicInteger();

    // Everything it needs comes in here: nothing is created inside
    RentalService(CarRepository cars, BookingRepository bookings, PricingStrategy pricing,
                  PaymentGateway payments, Clock clock) {
        this.cars = cars; this.bookings = bookings; this.pricing = pricing;
        this.payments = payments; this.clock = clock;
    }
```

One place, the **composition root**, decides which real classes to use. In a console app that's `main`:

```
    public static void main(String[] args) {
        CarRepository cars = new InMemoryCarRepository();
        cars.save(new Car("MH12-1001", "Swift", CarType.HATCHBACK, "Pune"));
        cars.save(new Car("MH12-2002", "City", CarType.SEDAN, "Pune"));
        cars.save(new Car("MH01-3003", "XUV700", CarType.SUV, "Mumbai"));
        PaymentGateway payments = (customer, amount) ->
                System.out.println("  (charged " + customer + " Rs " + amount + ")");
        Clock clock = Clock.fixed(Instant.parse("2026-12-01T04:30:00Z"), ZoneId.of("Asia/Kolkata"));
        RentalService service = new RentalService(cars, new InMemoryBookingRepository(),
                new PerDayPricing(), payments, clock);
```

In Spring, the container is the composition root: it calls this same constructor with the beans it finds. That's why constructor injection is the recommended style in Spring too; field injection with `@Autowired` on a private field hides what a class needs and can't be used without the container.

## What to inject, and what not to

| Inject | Don't inject (just create or pass as arguments) |
|---|---|
| Repositories · outside systems (payment, SMS, email, maps) · the clock · random or ID generators · strategies and policies | Entities and value objects (`Booking`, `DateRange`) · Java collections · constants |

## Fakes: small working stand-ins

In a test, pass in a **fake**: a tiny class that really works but does no harm. The in-memory repository is already one. `Clock.fixed(...)` is another, built into Java. For payments:

```
// A hand-written fake: records charges, and can be told to fail
class FakePaymentGateway implements PaymentGateway {
    final List<String> charges = new CopyOnWriteArrayList<>();
    boolean failNext;

    public void charge(String customer, int amount) {
        if (failNext) {
            failNext = false;
```

```
            throw new PaymentFailedException(customer);
        }
        charges.add(customer + ":" + amount);
    }
}
```

Then each test sets up a fresh service, does one thing and checks the result (arrange, act, assert):

```
    @BeforeEach
    void setUp() {
        CarRepository cars = new InMemoryCarRepository();
        cars.save(new Car("MH12-1001", "Swift", CarType.HATCHBACK, "Pune"));
        cars.save(new Car("MH12-2002", "City", CarType.SEDAN, "Pune"));
        payments = new FakePaymentGateway();
        Clock today = Clock.fixed(Instant.parse("2026-12-01T04:30:00Z"), ZoneId.of("Asia/Kolkata"));
        service = new RentalService(cars, new InMemoryBookingRepository(), new PerDayPricing(),
                payments, today);
    }

    @Test
    void aFailedPaymentLeavesTheCarFree() {
        payments.failNext = true;
        assertThrows(PaymentFailedException.class, () -> service.book("Asha", "MH12-2002", xmas));
        assertEquals(2, service.available("Pune", xmas).size());
    }
```

This is JUnit 5. The full test class, including a concurrency test, is in section 10; all seven tests pass. **Mocks** (Mockito) are the other option: you program a mock's answers and later check which calls it got. They're useful for "was the SMS sent exactly once?", but for LLD rounds a hand-written fake is clearer and shows your design really is swappable.

**Rules.** If a class creates what it depends on, it can't be tested without it. Pass it in through the constructor; let `main` (or Spring) pick the real ones. Inject time as a `Clock`. One behaviour per test, named as a sentence; every R line deserves at least one test, plus the edge cases.

**Common mistakes**

- `LocalDate.now()` deep inside business logic.
- `new SmsSender()` inside the service, so every test sends a real SMS, or can't run.
- Static maps and singletons that carry state from one test into the next.
- "Testing" by running `main` and reading the output by eye.
- Tests that use `Thread.sleep` and real time, so they're slow and sometimes fail.

**Say it like this.** "Everything the service talks to comes in through its constructor. In a test I pass an in-memory repository, a fixed clock and a fake payment gateway that I can tell to fail. Then I can check, say, that a failed payment leaves the car free, without a database or a real payment."

**Exercise 6** (25 min). A gym's renewal code. The rule: renewing adds one month, counted from today if the membership has already expired, otherwise from the current end date; the member gets an SMS.

```
class MembershipService {
    private final SmsSender sms = new SmsSender();                  // sends a real SMS

    LocalDate renew(String memberId) {
        Member m = Database.MEMBERS.get(memberId);                    // a static map
        LocalDate today = LocalDate.now();
        LocalDate start = m.expiry.isBefore(today) ? today : m.expiry;
        m.expiry = start.plusMonths(1);
        sms.send(m.phone, "Membership renewed till " + m.expiry);
        return m.expiry;
    }
}
```

**Part A.** List the three hidden dependencies, and one more problem.
**Part B.** Rewrite it with constructor injection (a `MemberRepository`, an `SmsSender` interface and a `Clock`), and write two JUnit tests with today fixed at 10 October 2026: a member who expired on 1 September renews to 10 November; a member whose plan ends on 25 October renews to 25 November and gets the SMS "Membership renewed till 2026-11-25".

# 7. Exceptions and error design

Every design has a "what if it goes wrong?" part: a car that doesn't exist, dates in the wrong order, a payment that fails. How you report those things shows whether you think about the **caller**, the code that has to deal with them.

## Checked or unchecked?

| | Checked (`extends Exception`) | Unchecked (`extends RuntimeException`) |
|---|---|---|
| **Compiler** | Forces every caller to catch it or declare `throws` | Doesn't force anything |
| **Meant for** | Problems outside your program that a caller can sensibly handle right there: a missing file, a dropped network connection | Bugs (null, bad index) and broken rules: bad input, wrong state, "car not available" |
| **In practice** | `IOException` at the edges of the program | Almost all domain errors in modern Java and Spring |

Two practical reasons for unchecked domain errors: checked exceptions don't work well inside lambdas and streams, and Spring's `@Transactional` rolls back on unchecked exceptions by default, but not on checked ones.

## Use the built-in ones first

| Exception | When | Example |
|---|---|---|
| `IllegalArgumentException` | The value passed in is wrong | return date before pickup date |
| `IllegalStateException` | The value is fine, but not now, in this state | cancelling a cancelled booking |
| `Optional.empty()` instead of an exception | "Not found" inside your own code, where the caller decides | `repository.findById` |

## When is a custom exception worth it?

When the caller will **react differently** to it, it needs to **map to a different HTTP status**, or it **carries data** the caller needs. "Car not available" is all three: the app shows other cars, the API answers 409 Conflict, and the message names the dates. Make one base class for the app and a few subclasses, not one per method:

```
// One base class for the whole app, a few subclasses the caller can react to
class WalletException extends RuntimeException {
    WalletException(String message) { super(message); }
    WalletException(String message, Throwable cause) { super(message, cause); }
}

class InsufficientFundsException extends WalletException {
    final int shortBy;
    InsufficientFundsException(String walletId, int amount, int balance) {
        super("Wallet " + walletId + " has Rs " + balance + ", needs Rs " + amount);
        this.shortBy = amount - balance;
    }
}

class AccountBlockedException extends WalletException {
    AccountBlockedException(String walletId) { super("Wallet " + walletId + " is blocked"); }
}

class Wallet {
    final String id;
    private int balance;
    private boolean blocked;

    Wallet(String id, int balance) { this.id = id; this.balance = balance; }

    void block() { blocked = true; }

    void withdraw(int amount) {
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive, was " + amount);
        if (blocked) throw new AccountBlockedException(id);
        if (amount > balance) throw new InsufficientFundsException(id, amount, balance);
        balance -= amount;
    }

    int balance() { return balance; }

    public static void main(String[] args) {
        Wallet w = new Wallet("W1", 500);
        w.withdraw(200);
        try {
            w.withdraw(450);
        } catch (InsufficientFundsException e) {
            System.out.println(e.getMessage() + " (short by Rs " + e.shortBy + ")");
        }
        try {
            w.withdraw(-5);
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
        w.block();
        try {
            w.withdraw(10);
        } catch (WalletException e) {
            System.out.println(e.getMessage());
        }
        System.out.println("Balance Rs " + w.balance());
    }
}
```

```
Wallet W1 has Rs 300, needs Rs 450 (short by Rs 150)
Amount must be positive, was -5
Wallet W1 is blocked
Balance Rs 300
```

Catching `WalletException` catches both subclasses; catching `InsufficientFundsException` lets the app offer a top-up of exactly `shortBy`. The second constructor keeps the original cause when you wrap a lower-level error.

## Throw, or return a result?

| Throw when… | Return a result when… |
|---|---|
| the caller broke the contract (bad input, unknown ID) | "failure" is a normal outcome the caller must handle every time |
| something went wrong that the normal flow doesn't expect (payment provider down) | there are several normal outcomes (joined, or waitlisted) |
| the code that can handle it is several calls up | you want to report many problems at once (a form with 3 bad fields) |
| *e.g.* `CarNotFoundException` | *e.g.* an empty search list · `tryAcquire()` returning `boolean` · a result type, like this: |

```
import java.util.*;

// Two normal outcomes, so a result type, not an exception
sealed interface JoinResult permits Joined, Waitlisted {}
record Joined(String className) implements JoinResult {}
record Waitlisted(String className, int position) implements JoinResult {}

class FitnessClass {
    final String name;
    final int capacity;
    private final List<String> members = new ArrayList<>(), waitlist = new ArrayList<>();

    FitnessClass(String name, int capacity) { this.name = name; this.capacity = capacity; }

    synchronized JoinResult join(String member) {
        if (members.contains(member) || waitlist.contains(member))
            throw new IllegalStateException(member + " is already in " + name);   // a caller bug
        if (members.size() < capacity) {
            members.add(member);
            return new Joined(name);
        }
        waitlist.add(member);
        return new Waitlisted(name, waitlist.size());
    }
}

public class Results {
    public static void main(String[] args) {
        FitnessClass zumba = new FitnessClass("Zumba 7AM", 2);
        for (String m : List.of("Asha", "Ravi", "Meera", "Kiran")) {
            String message = switch (zumba.join(m)) {          // the compiler checks every case
                case Joined j -> m + " joined " + j.className();
                case Waitlisted w -> m + " is number " + w.position() + " on the waitlist";
            };
            System.out.println(message);
        }
        try {
            zumba.join("Ravi");
        } catch (IllegalStateException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

```
Asha joined Zumba 7AM
Ravi joined Zumba 7AM
Meera is number 1 on the waitlist
Kiran is number 2 on the waitlist
Ravi is already in Zumba 7AM
```

The `sealed` interface lists every possible result, and the `switch` (Java 21) refuses to compile if a case is missing. Being waitlisted is a normal outcome, so it's a value; joining twice is a caller's bug, so it throws.

**Rules.** Unchecked for domain errors; checked only at the edges (files, network) where the caller can really do something. Built-ins first; a custom exception when the caller reacts differently. Messages name the IDs and values. Validate early, in constructors and at the start of methods. Never swallow an exception; when you wrap one, pass the cause. In machine coding, the driver catches and prints, as in every Phase 5 solution.

**Common mistakes**

- Returning `null`, `-1` or `-2` as error codes the caller has to remember.
- `catch (Exception e) { }`: the error disappears, and so does the bug.
- `throw new RuntimeException("error")`: no type to catch, no useful message.
- A custom exception for every method, or a checked one for every rule, so `throws` spreads everywhere.
- Using exceptions for normal flow, like ending a loop.

**Say it like this.** "Bad input is an IllegalArgumentException, a wrong state is an IllegalStateException. For the things a caller handles differently, I have a small hierarchy under RentalException, like CarNotAvailable, which the API would map to 409. Expected outcomes, like being waitlisted, are return values, not exceptions."

**Exercise 7** (15 min).
**Part A.** For each, choose: `IllegalArgumentException`, `IllegalStateException`, a custom exception (name it), `Optional`, an empty list, a `boolean`, a result type, or a checked exception.
1. `withdraw(-50)`. · 2. Cancelling a booking that's already cancelled. · 3. Searching Nashik for cars finds none. · 4. Booking a car that's taken; the app then suggests other cars. · 5. A rate limiter deciding whether a request may pass. · 6. Reading the pricing config file at startup; it's missing. · 7. Joining a full class puts you on a waitlist. · 8. An internal lookup by ID that may find nothing. · 9. The payment provider times out. · 10. A sign-up form where three of five fields are wrong, and the user should see all three.
**Part B.** Rewrite this:

```
int withdraw(String walletId, int amount) {     // 0 = ok, -1 = not enough money, -2 = blocked
    try {
        Wallet w = wallets.get(walletId);
        if (w.blocked) return -2;
        if (w.balance < amount) return -1;
        w.balance -= amount;
        return 0;
    } catch (Exception e) {
        return -3;
    }
}
```

# 8. Concurrency in LLD

> **The interviewer asks:** "Two users book the same seat at the same moment. What happens?"

This is the most common follow-up in booking, wallet, inventory and ride problems, and the most often hand-waved. You need a real answer: *which* lines race, *what* you lock, *why* that and not more, and what changes with several servers. You've already met every idea here in Phase 5; this section puts them in one place, with proof.

## 8.1 Bug one: the lost update

`count++` looks like one step. It's three: read `count`, add 1, write it back. Two threads can both read 41 and both write 42: one increment is lost.

```
public class LostUpdate {
    static int count = 0;

    public static void main(String[] args) throws Exception {
        Runnable work = () -> {
            for (int i = 0; i < 1_000_000; i++) count++;     // read, add 1, write back
        };
        Thread a = new Thread(work), b = new Thread(work);
        a.start(); b.start();
        a.join(); b.join();
        System.out.println("Expected 2000000, got " + count);
    }
}
```

**three real runs**

```
Expected 2000000, got 1705710
Expected 2000000, got 1953026
Expected 2000000, got 1972211
```

Hundreds of thousands of increments vanish, and the number is different every time. Fixes: make the method `synchronized`, or use an `AtomicInteger` and `incrementAndGet()`, which does all three steps as one.

## 8.2 Bug two: check-then-act

"If the seat is free, book it" is two steps. Two users can both see "free" before either writes "booked". This is the race behind every double booking.

```
import java.util.*;
import java.util.concurrent.*;

class Seat {
    private String bookedBy;                                  // null = free

    boolean bookUnsafe(String user) {
        if (bookedBy == null) {                               // check ...
            String ticket = makeTicket(user);                 // real work happens in the gap
            bookedBy = ticket;                                // ... then act
            return true;
        }
        return false;
    }

    synchronized boolean bookSafe(String user) {              // check and act: one step
        if (bookedBy == null) {
            bookedBy = makeTicket(user);
            return true;
        }
        return false;
    }

    // Stands in for the work between check and act: pricing, an ID, a database call
    private static String makeTicket(String user) {
        return UUID.randomUUID() + ":" + user;
    }
}

public class LastSeat {
    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(50);
        int unsafeTwice = 0, safeTwice = 0;
        for (int round = 0; round < 1000; round++) {
            if (winners(pool, new Seat(), false) > 1) unsafeTwice++;
            if (winners(pool, new Seat(), true) > 1) safeTwice++;
        }
        pool.shutdown();
        System.out.println("Unsafe: seat sold more than once in " + unsafeTwice + " of 1000 rounds");
        System.out.println("Safe:   seat sold more than once in " + safeTwice + " of 1000 rounds");
    }

    // 50 users try to book the same seat at the same moment; returns how many succeeded
    static int winners(ExecutorService pool, Seat seat, boolean safe) throws Exception {
        CountDownLatch go = new CountDownLatch(1);
        List<Future<Boolean>> results = new ArrayList<>();
        for (int i = 0; i < 50; i++) {
            String user = "user" + i;
            results.add(pool.submit(() -> {
                go.await();
                return safe ? seat.bookSafe(user) : seat.bookUnsafe(user);
            }));
        }
        go.countDown();
        int wins = 0;
        for (Future<Boolean> f : results) if (f.get()) wins++;
```

```
        return wins;
    }
}
```

**one real run**

```
Unsafe: seat sold more than once in 121 of 1000 rounds
Safe:   seat sold more than once in 0 of 1000 rounds
```

> **The honest numbers.** Across runs, the unsafe version sold the seat twice in anywhere from 7 to 121 rounds out of 1,000; the safe version never did. Without the "real work" in the gap it almost never failed here at all. That's the trap: a race can pass every test on your laptop and then sell a seat twice on a busy Friday. Tests can show a race exists; they can't prove there isn't one. Only reasoning about the code can.

## 8.3 The toolbox

| Tool | Use it for | It does *not*… |
|---|---|---|
| `synchronized` | Making a block one step for everyone locking the same object. The default choice. | help if two threads lock different objects |
| `volatile` | One thread writes a flag, others must see the new value ( `running = false` ) | make `count++` safe: it fixes visibility, not read-modify-write |
| `AtomicInteger` , `AtomicReference` | Counters, IDs, and claiming one thing with `compareAndSet` (Ride-sharing's drivers) | cover two variables at once |
| `ConcurrentHashMap` | A shared map; its single calls ( `merge` , `compute` , `putIfAbsent` , `computeIfAbsent` ) are atomic per key | make two calls in a row atomic (8.4) |
| `ReentrantLock` | What `synchronized` can't do: `tryLock` with a timeout, fairness, waiting that can be interrupted | unlock itself: always `unlock()` in `finally` |
| `ReadWriteLock` | Many readers, rare writers (a price list read all day, changed once) | pay off when writes are frequent |
| `ExecutorService` , `CountDownLatch` | Starting many threads together in a test (the rush helper) | protect anything by themselves |

## 8.4 The ConcurrentHashMap trap

A thread-safe map doesn't make *your* code thread-safe. Each call is atomic; the gap between two calls isn't.

```
import java.util.*;
import java.util.concurrent.*;

public class Votes {
    public static void main(String[] args) throws Exception {
        Map<String, Integer> wrong = new ConcurrentHashMap<>(), right = new ConcurrentHashMap<>();
        wrong.put("Modak", 0);
        ExecutorService pool = Executors.newFixedThreadPool(8);
        for (int t = 0; t < 8; t++) {
            pool.submit(() -> {
                for (int i = 0; i < 100_000; i++) {
                    wrong.put("Modak", wrong.get("Modak") + 1);       // two calls: a gap between them
                    right.merge("Modak", 1, Integer::sum);            // one atomic call
                }
            });
        }
        pool.shutdown();
        pool.awaitTermination(1, TimeUnit.MINUTES);
        System.out.println("Expected 800000");
        System.out.println("get then put: " + wrong.get("Modak"));
        System.out.println("merge:        " + right.get("Modak"));
    }
}
```

```
Expected 800000
get then put: 474227
merge:        800000
```

Half the votes for modak lost, from a "thread-safe" map. The same trap: `if (!map.containsKey(k)) map.put(k, …)` . Use the single atomic call: `merge` , `compute` , `putIfAbsent` or `computeIfAbsent` .

## 8.5 What to lock: the smallest thing that covers the check-then-act

| Choice | Phase 5 example | Good | Cost |
|---|---|---|---|
| **One lock for everything** | Food Delivery: every service method `synchronized` | simple, obviously correct | every request waits for every other one |
| **A lock per entity** | Hotel (per room), BookMyShow (per show), Inventory (per warehouse), section 10 (per car) | different rooms or cars never wait for each other | you must know which entity a request touches; two entities need an order (8.6) |
| **No lock: compare-and-set** | Ride-sharing: `status.compareAndSet(AVAILABLE, OFFERED)` | fastest; nobody waits | only for one variable; the loser must try something else |

> **Keep slow calls out of locks when you can.** Section 10 charges the payment inside the car's lock: correct, and other cars aren't affected, but a slow provider holds that car's lock for seconds. Real systems split it: hold the car (quick, under the lock), take payment (no lock), then confirm, with an expiry that releases unpaid holds. That's BookMyShow's design, and saying so is a strong answer.

## 8.6 Deadlock

Asha sends Ravi money while Ravi sends Asha money. Thread 1 locks Asha's wallet and waits for Ravi's; thread 2 locks Ravi's and waits for Asha's. Neither will ever let go. A deadlock needs all four of: locks that only one thread can hold, holding one while waiting for another, no way to take a lock away, and a circle of waiting. Break any one and it can't happen; the easy one to break is the circle.

```
import java.lang.management.*;

class Wallet {
    final String id;
    int balance = 1_000_000;
    Wallet(String id) { this.id = id; }
}

public class Deadlock {
    static void transferWrong(Wallet from, Wallet to, int amount) {
        synchronized (from) {                               // Asha->Ravi locks Asha first,
            synchronized (to) {                             // Ravi->Asha locks Ravi first
                from.balance -= amount;
                to.balance += amount;
            }
        }
    }

    static void transferRight(Wallet from, Wallet to, int amount) {
        // Always lock the smaller id first, whatever the direction
        Wallet first = from.id.compareTo(to.id) < 0 ? from : to;
        Wallet second = first == from ? to : from;
        synchronized (first) {
            synchronized (second) {
                from.balance -= amount;
                to.balance += amount;
            }
        }
    }

    public static void main(String[] args) throws Exception {
        boolean right = args.length > 0 && args[0].equals("right");
        Wallet asha = new Wallet("asha"), ravi = new Wallet("ravi");
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 100_000; i++)
                if (right) transferRight(asha, ravi, 1); else transferWrong(asha, ravi, 1);
        });
        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 100_000; i++)
                if (right) transferRight(ravi, asha, 1); else transferWrong(ravi, asha, 1);
        });
        t1.setDaemon(true); t2.setDaemon(true);
        t1.start(); t2.start();
        t1.join(3000); t2.join(3000);
        long[] stuck = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
        System.out.println(stuck == null
                ? "Finished: total " + (asha.balance + ravi.balance)
                : "Deadlock: " + stuck.length + " threads waiting for each other");
    }
}
```

```
$ java Deadlock
Deadlock: 2 threads waiting for each other
$ java Deadlock right
Finished: total 2000000
```

With the wrong order, 18 runs out of 20 deadlocked (Phase 5's wallet saw 10 of 30); with the same order every time, 0 of 10. `findDeadlockedThreads` is what `jstack` uses to report "Found one Java-level deadlock".

The other way out is to give up instead of waiting forever, with `ReentrantLock.tryLock`:

```
    // Give up instead of waiting forever; the caller can retry or report "busy"
    static boolean transfer(Account from, Account to, int amount) throws InterruptedException {
        if (!from.lock.tryLock(100, TimeUnit.MILLISECONDS)) return false;
        try {
            if (!to.lock.tryLock(100, TimeUnit.MILLISECONDS)) return false;
            try {
                if (from.balance < amount) return false;
                from.balance -= amount;
                to.balance += amount;
                return true;
            } finally {
                to.lock.unlock();                           // always, even if something throws
            }
        } finally {
            from.lock.unlock();
        }
    }
```

## 8.7 Pessimistic or optimistic?

| | Pessimistic: lock first, then work | Optimistic: work, then check nobody changed it |
|---|---|---|
| **In Java** | `synchronized`, `ReentrantLock` | `compareAndSet` in a retry loop |
| **In SQL** | `SELECT … FOR UPDATE` inside a transaction | `UPDATE … SET …, version = version + 1 WHERE id = ? AND version = ?`; 0 rows changed means someone was faster: reload and retry. JPA's `@Version` does this for you. |
| **Pick it when** | conflicts are common (the last seats at 10 AM Tatkal), or a retry is expensive | conflicts are rare (editing your own profile), and you don't want to hold locks |

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

record Balance(int amount, int version) {}

class OptimisticWallet {
    private final AtomicReference<Balance> state = new AtomicReference<>(new Balance(100_000, 0));
    final AtomicInteger retries = new AtomicInteger();

    // No lock: read, compute, and write only if nobody changed it meanwhile
    boolean pay(int amount) {
        while (true) {
            Balance now = state.get();
            if (now.amount() < amount) return false;
            Balance next = new Balance(now.amount() - amount, now.version() + 1);
            if (state.compareAndSet(now, next)) return true;      // like UPDATE ... WHERE version = ?
            retries.incrementAndGet();                            // someone was faster: try again
        }
    }

    Balance balance() { return state.get(); }
}

public class Optimistic {
    public static void main(String[] args) throws Exception {
        OptimisticWallet w = new OptimisticWallet();
        ExecutorService pool = Executors.newFixedThreadPool(8);
        for (int t = 0; t < 8; t++)
            pool.submit(() -> { for (int i = 0; i < 10_000; i++) w.pay(1); });
        pool.shutdown();
        pool.awaitTermination(1, TimeUnit.MINUTES);
        System.out.println("Balance " + w.balance().amount() + " (expected 20000), version "
                + w.balance().version() + ", retries " + w.retries.get());
    }
}
```

```
Balance 20000 (expected 20000), version 80000, retries 8555
```

The total is exactly right with no lock at all; thousands of retries show how often two threads collided. With heavy contention those retries become the cost, which is when pessimistic locking wins.

**A third database option: the conditional update.** `UPDATE seat SET status = 'HELD' WHERE id = ? AND status = 'FREE'`. One row changed: it's yours. Zero: someone else got it. The check and the act are one statement, so the database does the locking. It's often the best answer for "how do you book safely with a database?"

## 8.8 More than one server

`synchronized` works inside one JVM. Put the app on three servers and two of them can book the same car at the same moment. Options, best first: let the **database** decide (a conditional update, a unique constraint, or a row lock); **route by key**, so all requests for one car go to the same server; or a **distributed lock** (Redis `SET key value NX PX 30000`), which needs an expiry in case the holder dies, and care when the expiry runs out mid-work.

## 8.9 How to test it

Start many threads together (the rush helper, or a `CountDownLatch`), make them fight over one thing, and check the invariant: one winner, stock never below zero, totals unchanged. Run it many times. Then remove your lock and check the test fails: section 10's test failed in 6 runs out of 40 without the car lock. A test that never fails, even when the lock is gone, isn't testing anything.

## 8.10 The answer, in five steps

| | |
|---|---|
| **1** | What is shared and changes? (the seats of a show, a driver's status, a wallet's balance) |
| **2** | Where is the check-then-act, or read-modify-write? (is it free → book it) |
| **3** | What's the smallest thing that covers it? Lock that, or use one atomic call. |
| **4** | Two locks at once? Always take them in the same order. |
| **5** | Several servers? Move the guarantee into the database. |

**Common mistakes**

- Locking on different objects: `synchronized (new Object())`, or an instance method guarding a `static` field.
- `volatile int count; count++;`: still a lost update.
- `get` then `put` on a `ConcurrentHashMap`.
- Holding a lock while calling a slow outside service, so everyone queues behind it.
- `synchronized` on everything "to be safe": slow, and the more locks, the more chances of deadlock.
- A `ReentrantLock` unlocked outside `finally`: one exception and it stays locked forever.

**Say it like this.** "The race is between checking the car is free and saving the booking: two requests can both see it free. I'll make those one step with a lock per car, so bookings for other cars don't wait. If a transfer ever needs two locks, I'll take them in ID order to avoid deadlock. With several servers, synchronized isn't enough, so I'd move the check into the database as a conditional update."

**Exercise 8** (30 min).
**Part A.** Each snippet runs on many threads. What's wrong, and what's the fix?

```
(a) Map<String, List<String>> byCity = new ConcurrentHashMap<>();
    if (!byCity.containsKey(city)) byCity.put(city, new ArrayList<>());
    byCity.get(city).add(car);

(b) private volatile int visitors;
    void enter() { visitors++; }

(c) void sell() { synchronized (new Object()) { if (stock > 0) stock--; } }

(d) class Counter {
        private static int total;
        synchronized void add() { total++; }      // many Counter objects exist
    }

(e) synchronized void book(String seat) {        // one lock for the whole theatre
        check(seat); payment.charge(...);          // charge() takes about 3 seconds
        save(seat);
    }

(f) void transfer(Wallet from, Wallet to, int amt) {
        synchronized (from) { synchronized (to) { ... } }
    }
```

**Part B.** Ganpati festival darshan passes: each time slot has a fixed number of passes, and a person can hold at most one pass per slot. This works with one counter and breaks under load. Find the races, then fix it so that 2,000 people rushing a 500-pass slot get exactly 500 passes, and Asha tapping "book" 20 times at once gets one. Slots must not wait for each other.

```
import java.util.*;

// As given: works for one counter, breaks when many people book at once
class PassBooking {
    private final Map<String, Integer> capacity = new HashMap<>();      // slot -> passes
    private final Map<String, Set<String>> holders = new HashMap<>();   // slot -> people

    void addSlot(String slot, int passes) {
        capacity.put(slot, passes);
        holders.put(slot, new HashSet<>());
    }

    boolean book(String slot, String person) {
        Set<String> people = holders.get(slot);
        if (people.size() >= capacity.get(slot)) return false;          // full
        if (people.contains(person)) return false;                      // one pass per person
        people.add(person);
        return true;
    }
}
```

**Part C.** Pessimistic or optimistic, and why? (1) Users editing their own profile. (2) The last 20 seats of a Tatkal train at 10:00 AM. (3) Two editors changing the same shared document. (4) A wallet debited many times a second by one busy merchant.

## 9. Thinking out loud while designing

This trips up more candidates than the code does. The interviewer grades your **reasoning**, and they can only grade what they hear. Silence looks the same whether you're stuck or thinking hard. Talking through a trade-off sounds competent even when you haven't decided yet.

### A plan for the round

| LLD discussion (45 min) | You do | You say, for example |
|---|---|---|
| 0–5 | Requirements: R, NF, out of scope | "Let me take four minutes on scope first." |
| 5–12 | Entities and a diagram | "These are my entities; DateRange is a value object because…" |
| 12–15 | Name what varies; pick interfaces | "Pricing will change, so it's a strategy. Notifications are listeners." |
| 15–35 | Code the core flow first | "I'll write booking end to end first, then cancel." |
| 35–45 | Follow-ups: concurrency, failure, scale, a change | "The race is between the check and the save…" |

| Machine coding (90 min) | You do |
|---|---|
| 0–10 | Read everything; write assumptions as a comment at the top; plan the classes |
| 10–25 | Model, repositories, the service skeleton; get it compiling and running with one command |
| 25–70 | Core flows one at a time, running after each one. Working code at every step beats a perfect design that doesn't run. |
| 70–80 | Run all the given input; fix the output to match exactly |
| 80–90 | Edge cases, then the extension they asked for; clean up names |

## Six kinds of sentence to have ready

| Kind | Example |
|---|---|
| **Announce** | "I'll start with the entities, then the booking flow, then concurrency." |
| **Options and a choice** | "Two ways: one lock for everything, which is simple but makes every booking wait, or a lock per car. I'll take per car, because bookings for different cars have nothing to do with each other." |
| **Assume** | "I'll assume rentals are by day. If hours matter, only DateRange and pricing change." |
| **Stuck** | "Let me think about the overlap condition for half a minute." Then think on paper, where they can see it. |
| **Check in** | "Should I code cancel next, or would you rather talk about concurrency?" |
| **Push-back** | "You'd be right if pricing never changed. Here it changes every season, so the interface saves us edits; if it really never changes, I'd inline it." |

The push-back sentence is the same shape as every amber box in Phase 5: agree with what's true in their point, say what's different here, and say when you'd switch.

**Rules.** Say *why*, not *what*: the interviewer can see you're writing a loop; they can't see why you chose a TreeMap. Every decision gets a reason and, ideally, the option you rejected. No silence longer than about 30 seconds without saying what you're thinking about. Take hints: if the interviewer asks "what if two people...?", they're telling you what matters to them.

**Common mistakes**

- Narrating typing: "now I'm creating a HashMap... now a for loop..."
- Two minutes of silence, then a finished answer with no reasoning shown.
- Pattern names with no reason: "I'll use Strategy here."
- "It will scale", "it's thread-safe", with no how.
- Defending every choice; a concession plus a reason sounds stronger than a fight.

**Practice drill.** Pick a Phase 5 problem you've already solved. Set a 10-minute timer, record yourself on your phone doing the requirements, entities and the core flow out loud, then listen. Count the silences longer than 20 seconds, and the decisions you made without a reason. Do it again a week later and compare.

**Exercise 9** (10 min).
**Part A.** Rewrite each into something worth hearing:
1. *(two minutes of silence while working out how to find overlapping bookings)*
2. "I'll use the Strategy pattern."
3. "It will scale."
4. "Now I'm making a HashMap, now I'm writing a loop over it..."
5. Interviewer: "Why not just one lock for everything?" You: "No, that's bad."

**Part B.** Write the first 60 seconds you'd say after "Design an Amazon Locker."

# 10. Everything together: a car rental service

The requirements are the R and NF lines from section 1. Here is the whole thing as one runnable file, in layers, followed by its tests. Read it once with this map in hand:

| Section | Where it shows up in the code |
|---|---|
| 2. Entities | `Car` and `Booking` are entities; `DateRange` is a value object (a record that checks itself); `CarType` is an enum because types differ only in price. |
| 3. UML | The diagram in section 3 is exactly these classes. |
| 4. Layers | Comment lines split model, exception, repository, service and wiring. `Booking.cancel()` guards its own state; the overlap rule is in the service; `main` only wires and prints. |
| 5. Repositories | Two interfaces, two in-memory classes, `Optional` for one result, copies for lists, an index by car. |
| 6. Injection | `RentalService` gets everything through its constructor, including a `Clock`; `main` is the only place that picks real classes. |
| 7. Exceptions | Built-ins for bad input and wrong state; `RentalException` and three subclasses for what callers handle differently. |
| 8. Concurrency | One lock per car around check, price, charge and save; bookings for other cars never wait. |

## The code

```
import java.time.*;
import java.time.temporal.ChronoUnit;
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

// ================ model ================
enum CarType {
    HATCHBACK(1500), SEDAN(2200), SUV(3000);

    final int perDay;
    CarType(int perDay) { this.perDay = perDay; }
}

record Car(String id, String model, CarType type, String city) {}

// A value object: no ID, never changes, two are equal if their fields are equal
record DateRange(LocalDate from, LocalDate to) {
    DateRange {
        if (!to.isAfter(from))
            throw new IllegalArgumentException("Return date must be after pickup date");
    }
    boolean overlaps(DateRange o) { return from.isBefore(o.to) && o.from.isBefore(to); }
    long days() { return ChronoUnit.DAYS.between(from, to); }
}

enum BookingStatus { BOOKED, CANCELLED }

class Booking {
    final String id, carId, customer;
    final DateRange range;
    final int amount;
    private BookingStatus status = BookingStatus.BOOKED;

    Booking(String id, String carId, String customer, DateRange range, int amount) {
        this.id = id; this.carId = carId; this.customer = customer;
        this.range = range; this.amount = amount;
    }

    boolean isActive() { return status == BookingStatus.BOOKED; }

    // The entity guards its own rule
    void cancel() {
        if (status != BookingStatus.BOOKED)
            throw new IllegalStateException(id + " is already cancelled");
        status = BookingStatus.CANCELLED;
    }
}

// ================ exception ================
class RentalException extends RuntimeException {
    RentalException(String message) { super(message); }
}

class CarNotFoundException extends RentalException {
    CarNotFoundException(String carId) { super("No car " + carId); }
}

class CarNotAvailableException extends RentalException {
    CarNotAvailableException(String carId, DateRange r) {
        super("Car " + carId + " is not free from " + r.from() + " to " + r.to());
    }
}

class PaymentFailedException extends RentalException {
    PaymentFailedException(String customer) { super("Payment failed for " + customer); }
}

// ================ repository ================
interface CarRepository {
    void save(Car car);
    Optional<Car> findById(String id);
```

```
    List<Car> findByCity(String city);
}

interface BookingRepository {
    void save(Booking booking);
    Optional<Booking> findById(String id);
    List<Booking> findByCar(String carId);
}

class InMemoryCarRepository implements CarRepository {
    private final Map<String, Car> cars = new ConcurrentHashMap<>();

    public void save(Car car) { cars.put(car.id(), car); }
    public Optional<Car> findById(String id) { return Optional.ofNullable(cars.get(id)); }

    public List<Car> findByCity(String city) {
        return cars.values().stream().filter(c -> c.city().equals(city))
                .sorted(Comparator.comparing(Car::id)).toList();
    }
}

class InMemoryBookingRepository implements BookingRepository {
    private final Map<String, Booking> byId = new ConcurrentHashMap<>();
    private final Map<String, List<Booking>> byCar = new ConcurrentHashMap<>();    // an index

    public void save(Booking b) {
        if (byId.put(b.id, b) == null)                          // new, not an update
            byCar.computeIfAbsent(b.carId, k -> new CopyOnWriteArrayList<>()).add(b);
    }

    public Optional<Booking> findById(String id) { return Optional.ofNullable(byId.get(id)); }

    public List<Booking> findByCar(String carId) {
        return List.copyOf(byCar.getOrDefault(carId, List.of()));    // a copy, not our list
    }
}

// ================= service =================
interface PricingStrategy {
    int price(Car car, DateRange range);
}

class PerDayPricing implements PricingStrategy {
    public int price(Car car, DateRange range) { return car.type().perDay * (int) range.days(); }
}

interface PaymentGateway {
    void charge(String customer, int amount);                   // throws PaymentFailedException
}

class RentalService {
    private final CarRepository cars;
    private final BookingRepository bookings;
    private final PricingStrategy pricing;
    private final PaymentGateway payments;
    private final Clock clock;
    private final Map<String, Object> carLocks = new ConcurrentHashMap<>();
    private final AtomicInteger count = new AtomicInteger();

    // Everything it needs comes in here: nothing is created inside
    RentalService(CarRepository cars, BookingRepository bookings, PricingStrategy pricing,
                  PaymentGateway payments, Clock clock) {
        this.cars = cars; this.bookings = bookings; this.pricing = pricing;
        this.payments = payments; this.clock = clock;
    }

    List<Car> available(String city, DateRange range) {
        return cars.findByCity(city).stream().filter(c -> isFree(c.id(), range)).toList();
    }

    Booking book(String customer, String carId, DateRange range) {
        if (range.from().isBefore(LocalDate.now(clock)))
            throw new IllegalArgumentException("Pickup date " + range.from() + " is in the past");
        Car car = cars.findById(carId).orElseThrow(() -> new CarNotFoundException(carId));
        // One car at a time; bookings for other cars don't wait
        synchronized (lockFor(carId)) {
            if (!isFree(carId, range)) throw new CarNotAvailableException(carId, range);
            int amount = pricing.price(car, range);
            payments.charge(customer, amount);                  // if this throws, nothing is saved
            Booking b = new Booking("B" + count.incrementAndGet(), carId, customer, range, amount);
            bookings.save(b);
            return b;
        }
    }

    Booking cancel(String bookingId) {
        Booking b = bookings.findById(bookingId)
                .orElseThrow(() -> new IllegalArgumentException("No booking " + bookingId));
        synchronized (lockFor(b.carId)) {
            b.cancel();
            bookings.save(b);                                   // a real database needs this
        }
        return b;
    }

    private Object lockFor(String carId) { return carLocks.computeIfAbsent(carId, k -> new Object()); }

    private boolean isFree(String carId, DateRange range) {
        return bookings.findByCar(carId).stream()
                .noneMatch(b -> b.isActive() && b.range.overlaps(range));
    }
```

```
}

// ================ wiring: the only place that picks implementations ================
public class Rental {
    public static void main(String[] args) {
        CarRepository cars = new InMemoryCarRepository();
        cars.save(new Car("MH12-1001", "Swift", CarType.HATCHBACK, "Pune"));
        cars.save(new Car("MH12-2002", "City", CarType.SEDAN, "Pune"));
        cars.save(new Car("MH01-3003", "XUV700", CarType.SUV, "Mumbai"));
        PaymentGateway payments = (customer, amount) ->
                System.out.println("  (charged " + customer + " Rs " + amount + ")");
        Clock clock = Clock.fixed(Instant.parse("2026-12-01T04:30:00Z"), ZoneId.of("Asia/Kolkata"));
        RentalService service = new RentalService(cars, new InMemoryBookingRepository(),
                new PerDayPricing(), payments, clock);

        DateRange xmas = new DateRange(LocalDate.of(2026, 12, 24), LocalDate.of(2026, 12, 27));
        run(() -> {
            Booking b = service.book("Asha", "MH12-2002", xmas);
            System.out.println(b.id + ": " + b.customer + ", " + b.carId + ", Rs " + b.amount);
        });
        run(() -> service.book("Ravi", "MH12-2002", xmas));
        run(() -> service.book("Ravi", "MH12-9999", xmas));
        run(() -> service.book("Ravi", "MH12-1001",
                new DateRange(LocalDate.of(2026, 11, 20), LocalDate.of(2026, 11, 22))));
        run(() -> new DateRange(LocalDate.of(2026, 12, 24), LocalDate.of(2026, 12, 24)));
        run(() -> System.out.println("Free in Pune: " + service.available("Pune", xmas).stream()
                .map(Car::model).toList()));
        run(() -> System.out.println("Cancelled " + service.cancel("B1").id));
        run(() -> service.cancel("B1"));
        run(() -> System.out.println("Free in Pune: " + service.available("Pune", xmas).stream()
                .map(Car::model).toList()));
    }

    // The driver catches what the service throws and prints the message
    static void run(Runnable action) {
        try {
            action.run();
        } catch (RentalException | IllegalArgumentException | IllegalStateException e) {
            System.out.println(e.getClass().getSimpleName() + ": " + e.getMessage());
        }
    }
}
```

**output of main**

```
  (charged Asha Rs 6600)
B1: Asha, MH12-2002, Rs 6600
CarNotAvailableException: Car MH12-2002 is not free from 2026-12-24 to 2026-12-27
CarNotFoundException: No car MH12-9999
IllegalArgumentException: Pickup date 2026-11-20 is in the past
IllegalArgumentException: Return date must be after pickup date
Free in Pune: [Swift]
Cancelled B1
IllegalStateException: B1 is already cancelled
Free in Pune: [Swift, City]
```

## The tests (JUnit 5)

```
import static org.junit.jupiter.api.Assertions.*;

import java.time.*;
import java.util.*;
import java.util.concurrent.*;
import org.junit.jupiter.api.*;

// A hand-written fake: records charges, and can be told to fail
class FakePaymentGateway implements PaymentGateway {
    final List<String> charges = new CopyOnWriteArrayList<>();
    boolean failNext;

    public void charge(String customer, int amount) {
        if (failNext) {
            failNext = false;
            throw new PaymentFailedException(customer);
        }
        charges.add(customer + ":" + amount);
    }
}

class RentalServiceTest {
    private final DateRange xmas =
            new DateRange(LocalDate.of(2026, 12, 24), LocalDate.of(2026, 12, 27));
    private FakePaymentGateway payments;
    private RentalService service;

    @BeforeEach
    void setUp() {
        CarRepository cars = new InMemoryCarRepository();
        cars.save(new Car("MH12-1001", "Swift", CarType.HATCHBACK, "Pune"));
        cars.save(new Car("MH12-2002", "City", CarType.SEDAN, "Pune"));
        payments = new FakePaymentGateway();
        Clock today = Clock.fixed(Instant.parse("2026-12-01T04:30:00Z"), ZoneId.of("Asia/Kolkata"));
        service = new RentalService(cars, new InMemoryBookingRepository(), new PerDayPricing(),
                payments, today);
    }

    @Test
    void booksAFreeCarAndChargesForEachDay() {
```

```
        Booking b = service.book("Asha", "MH12-2002", xmas);
        assertEquals(6600, b.amount);                                   // 3 days x Rs 2200
        assertEquals(List.of("Asha:6600"), payments.charges);
    }

    @Test
    void refusesAnOverlappingBooking() {
        service.book("Asha", "MH12-2002", xmas);
        DateRange overlap = new DateRange(LocalDate.of(2026, 12, 26), LocalDate.of(2026, 12, 28));
        assertThrows(CarNotAvailableException.class, () -> service.book("Ravi", "MH12-2002", overlap));
    }

    @Test
    void allowsPickupOnTheDayTheLastRenterReturns() {
        service.book("Asha", "MH12-2002", xmas);
        DateRange next = new DateRange(LocalDate.of(2026, 12, 27), LocalDate.of(2026, 12, 29));
        assertEquals(4400, service.book("Ravi", "MH12-2002", next).amount);
    }

    @Test
    void refusesAPickupDateInThePast() {
        DateRange past = new DateRange(LocalDate.of(2026, 11, 20), LocalDate.of(2026, 11, 22));
        assertThrows(IllegalArgumentException.class, () -> service.book("Asha", "MH12-1001", past));
    }

    @Test
    void aFailedPaymentLeavesTheCarFree() {
        payments.failNext = true;
        assertThrows(PaymentFailedException.class, () -> service.book("Asha", "MH12-2002", xmas));
        assertEquals(2, service.available("Pune", xmas).size());
    }

    @Test
    void cancellingFreesTheCar() {
        Booking b = service.book("Asha", "MH12-2002", xmas);
        service.cancel(b.id);
        assertEquals(2, service.available("Pune", xmas).size());
        assertThrows(IllegalStateException.class, () -> service.cancel(b.id));
    }

    @Test
    void onlyOneOfFiftySimultaneousBookingsWins() throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(50);
        CountDownLatch go = new CountDownLatch(1);
        List<Future<Boolean>> results = new ArrayList<>();
        for (int i = 0; i < 50; i++) {
            String customer = "user" + i;
            results.add(pool.submit(() -> {
                go.await();
                try {
                    service.book(customer, "MH12-2002", xmas);
                    return true;
                } catch (CarNotAvailableException e) {
                    return false;
                }
            }));
        }
        go.countDown();
        int wins = 0;
        for (Future<Boolean> f : results) if (f.get()) wins++;
        pool.shutdown();
        assertEquals(1, wins);
        assertEquals(1, payments.charges.size());
    }
}
```

**Results.** All 7 tests pass on every run. To check the concurrency test is worth having, the car lock was removed: the last test then failed in **6 runs out of 40**, with two customers charged for the same car. To run them in IntelliJ, add JUnit 5 ( `org.junit.jupiter:junit-jupiter` ) to the project and put each class in its own file.

## Three follow-ups to answer out loud

1. **"Payment is slow. What happens to other customers?"** Only people booking *the same car* wait, because the lock is per car. To stop even that, hold the car first, take payment outside the lock, then confirm; release holds that aren't paid within 10 minutes.
2. **"Weekend rates and a 10% discount over 7 days."** A new `PricingStrategy` , or a decorator around `PerDayPricing` , passed in by `main` . The service doesn't change.
3. **"Three servers and MySQL."** `JdbcBookingRepository` replaces the in-memory one, and the per-car lock moves into the database: lock the car's row with `SELECT … FOR UPDATE` inside the booking transaction, or insert with a constraint that refuses overlapping ranges.

# Answers

## Exercise 1 — requirements

**Part A** (any six like these). *Actors:* members and trainers only, or branch admins too? *Core flows:* book, cancel, see the timetable; is buying a plan in scope? *Rules:* how many classes a week does each plan allow? Until when can a member cancel? Is there a waitlist when a class is full? Can a member book at any branch? *Edge cases:* two classes at the same time? A class after the membership ends? *Non-functional:* many members booking the 7 AM class the moment it opens? Which rules change most? *Scope:* can I skip login and the UI?

| # | Part B | Part C: what it changes in the design |
|---|---|---|
| 1 | F | |
| 2 | NF | Checking for a free spot and taking it are one step: a lock per class |
| 3 | OUT | (the interviewer said so; at most a `PaymentGateway` interface) |
| 4 | F | |
| 5 | NF | Plans as data (an enum or records with price and classes per week); an interface only if their rules differ |
| 6 | F | (a scheduled job, plus an `SmsSender` interface) |
| 7 | NF | Map lookups and indexes instead of scanning everything. Say it; there's little to draw. |
| 8 | F | A rule with a number is still functional. Needs an injected `Clock` to test. |
| 9 | NF | Repository interfaces with in-memory classes |
| 10 | F | |
| 11 | NF | Never delete bookings: change their status, and keep an append-only list of events |
| 12 | NF | Constructor injection; fakes for repositories, SMS and the clock |

## Exercise 2 — entities

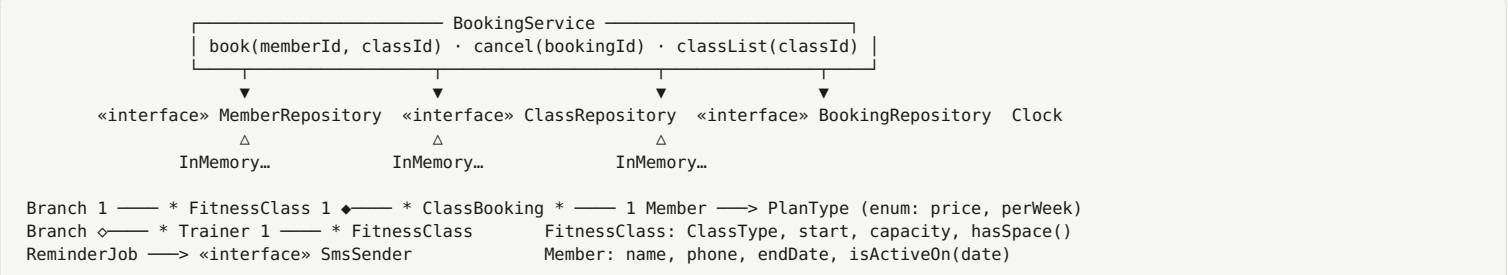
| Noun | Decision |
|---|---|
| gym chain | The system itself: not a class (at most the service is its front door) |
| branch, class, member, trainer | Entities. Call the class `FitnessClass`, not `Class`, which clashes with `java.lang.Class`. |
| booking | An entity hidden in a verb ("members *book*"). The most important noun on the list, and the one most people miss. |
| Zumba, yoga, spinning | An enum `ClassType` (or just a name): they differ only in name |
| start time, number of spots, membership end date | Attributes (`LocalDateTime`, `int capacity`, `LocalDate`) |
| plan: monthly, quarterly, student | An enum with fields (price, classes a week): they differ in data. An interface only if their *rules* differ. |
| trainer (as a user), member (as a user) | Also actors |
| SMS reminder | An outside system: an `SmsSender` interface, called by a scheduled job |

**Part B.** (a) `FitnessClass.hasSpace()`: its own capacity and bookings. (b) The plan: `PlanType.STUDENT.price`, or a pricing strategy if discounts keep changing. (c) `ClassBooking.cancel()` changes its own status; the service checks the 2-hour rule with the clock and frees the spot. (d) `Member.isActiveOn(date)`: its own end date. (e) A reminder job using `SmsSender`: time passing isn't an event, so a scheduled job, not an Observer. (f) `BookingService.book(memberId, classId)`: it needs the member (active? weekly limit?), the class (space?) and the repository, under a lock per class.

## Exercise 3 — UML

**Part A.** (1) `Branch 1 ——— * FitnessClass` · (2) `StudentPlan - - -▷ «interface» Plan` · (3) `FitnessClass 1 ◆——— * ClassBooking` · (4) `Trainer 1 ——— * FitnessClass`, and `Branch ◇——— * Trainer` because a trainer can move · (5) `BookingService - - -> LocalDate` (a parameter only: a dependency) · (6) `PremiumMember ——▷ Member`.

**Part B** (one good answer):



## Exercise 4 — layers

| # | Belongs in | What's wrong now |
|---|---|---|
| 1 | Repository | A public static map is global state: shared between tests, not thread-safe, can't be swapped. Use an injected `ClassRepository`. |
| 2 | Driver | Parsing input is the driver's job; the service should take `(memberId, classId)`. |
| 3 | Entity | A rule about the class's own state: `FitnessClass.book(member)` should check it. |
| 4, 7 | Driver | The service should throw (or return a result); only the driver prints. |
| 5 | Strategy / plan | A pricing rule that will change, hard-coded in the middle of booking. |
| 6 | Entity | Outside code changing the class's internal list. With (3), it becomes one method on `FitnessClass`, which can then be one locked step. Right now, two members can take the last spot. |

## Exercise 5 — repositories

**Part A.** (a) Public, static and a plain `HashMap`: anyone can change it, tests share it, threads corrupt it. Make it a `private final ConcurrentHashMap` inside `InMemoryMemberRepository`, passed to whoever needs it. (b) Returns `null`: return `Optional.ofNullable(members.get(id))`. (c) Hands out the internal map: return `List.copyOf(members.values())`. (d) Keyed by name, so a second Asha replaces the first: key by ID. (e) A business rule in the repository: renewing belongs in the service; the repository only finds and saves.

**Part B.**

```
interface ClassBookingRepository {
    void save(ClassBooking booking);                              // new or changed
    Optional<ClassBooking> findById(String bookingId);
    List<ClassBooking> findByClass(String classId);                // who booked this class?
    List<ClassBooking> findByMemberBetween(String memberId, LocalDate from, LocalDate to);
}
```

## Exercise 6 — injection and tests

**Part A.** (1) `new SmsSender()`: every test sends a real SMS. (2) `Database.MEMBERS`: a static map, shared by all tests and impossible to swap for MySQL. (3) `LocalDate.now()`: the result depends on the day the test runs. One more: an unknown ID gives a `NullPointerException` instead of a clear error, and nothing calls `save()` after the change.

**Part B.**

```
import java.time.*;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

class Member {
    final String id, phone;
    LocalDate expiry;
    Member(String id, String phone, LocalDate expiry) {
        this.id = id; this.phone = phone; this.expiry = expiry;
    }
}

interface MemberRepository {
    Optional<Member> findById(String id);
    void save(Member m);
}

class InMemoryMemberRepository implements MemberRepository {
    private final Map<String, Member> members = new ConcurrentHashMap<>();
    public Optional<Member> findById(String id) { return Optional.ofNullable(members.get(id)); }
    public void save(Member m) { members.put(m.id, m); }
}

interface SmsSender {
    void send(String phone, String text);
}

class MembershipService {
    private final MemberRepository members;
    private final SmsSender sms;
    private final Clock clock;

    MembershipService(MemberRepository members, SmsSender sms, Clock clock) {
        this.members = members; this.sms = sms; this.clock = clock;
    }

    LocalDate renew(String memberId) {
        Member m = members.findById(memberId)
                .orElseThrow(() -> new IllegalArgumentException("No member " + memberId));
        LocalDate today = LocalDate.now(clock);
        LocalDate start = m.expiry.isBefore(today) ? today : m.expiry;
        m.expiry = start.plusMonths(1);
        members.save(m);
        sms.send(m.phone, "Membership renewed till " + m.expiry);
        return m.expiry;
    }
}
```

```
import static org.junit.jupiter.api.Assertions.*;

import java.time.*;
import java.util.*;
import org.junit.jupiter.api.*;

class FakeSms implements SmsSender {
```

```
    final List<String> sent = new ArrayList<>();
    public void send(String phone, String text) { sent.add(phone + ": " + text); }
}

class MembershipServiceTest {
    private final Clock oct10 =
            Clock.fixed(Instant.parse("2026-10-10T03:30:00Z"), ZoneId.of("Asia/Kolkata"));
    private InMemoryMemberRepository repo;
    private FakeSms sms;
    private MembershipService service;

    @BeforeEach
    void setUp() {
        repo = new InMemoryMemberRepository();
        sms = new FakeSms();
        service = new MembershipService(repo, sms, oct10);
    }

    @Test
    void anExpiredMemberRenewsFromToday() {
        repo.save(new Member("M1", "98220", LocalDate.of(2026, 9, 1)));
        assertEquals(LocalDate.of(2026, 11, 10), service.renew("M1"));
    }

    @Test
    void anActiveMemberRenewsFromTheCurrentExpiryAndGetsAnSms() {
        repo.save(new Member("M2", "98221", LocalDate.of(2026, 10, 25)));
        assertEquals(LocalDate.of(2026, 11, 25), service.renew("M2"));
        assertEquals(List.of("98221: Membership renewed till 2026-11-25"), sms.sent);
    }
}
```

Both tests pass. Today is fixed with `Clock.fixed`, so they give the same result on any day.

## Exercise 7 — exceptions

| # | Answer | Why |
|---|---|---|
| 1 | `IllegalArgumentException` | the value passed in is wrong |
| 2 | `IllegalStateException` | a fine request at the wrong time |
| 3 | an empty list | "nothing found" is a normal search result |
| 4 | `CarNotAvailableException` | the caller reacts differently (suggests other cars), and the API maps it to 409 |
| 5 | `boolean` | both answers are normal; the caller branches every time |
| 6 | checked `IOException`, at startup | a real outside problem; wrap it in an unchecked config error with the cause and stop the app early |
| 7 | a result type (Joined / Waitlisted) | two normal outcomes |
| 8 | `Optional` | the caller decides whether missing is an error |
| 9 | `PaymentFailedException`, wrapping the timeout as its cause | the caller may retry or tell the user; the cause helps debugging |
| 10 | a result with a list of errors | an exception would stop at the first bad field |

**Part B.** Error codes become exceptions with messages; the `catch (Exception e)` goes, because it was hiding real bugs (an unknown wallet ID threw a `NullPointerException` and came back as "-3"). The service finds the wallet with `orElseThrow`, and the wallet guards its own rules:

```
    void withdraw(int amount) {
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive, was " + amount);
        if (blocked) throw new AccountBlockedException(id);
        if (amount > balance) throw new InsufficientFundsException(id, amount, balance);
        balance -= amount;
    }
```

## Exercise 8 — concurrency

**Part A.** (a) Three separate calls: two threads can both create a list, and one list (with its cars) is lost; and `ArrayList` isn't thread-safe anyway. Fix: `byCity.computeIfAbsent(city, k -> new CopyOnWriteArrayList<>()).add(car);` (b) `volatile` doesn't make `++` one step: use an `AtomicInteger` and `incrementAndGet()`. (c) Every call locks its own brand-new object, so nothing is shared: lock a `private final Object lock` field, or make the method `synchronized`. (d) An instance lock guarding a `static` field: two `Counter` objects lock different things. Use `static synchronized`, or a static `AtomicInteger`. (e) Correct, but everyone in the theatre waits three seconds behind each payment: lock per show (or seat), and take payment outside the lock (hold, pay, confirm). (f) Two opposite transfers deadlock: lock the two wallets in ID order, or use `tryLock` with a timeout.

**Part B.** Three problems in the given code: (1) "is it full?" then add: two threads both see 499 and both add, giving 501 passes; (2) "does Asha have one?" then add: two taps both see no; (3) `HashSet` and `HashMap` aren't safe for threads at all, so concurrent adds can corrupt them. Run 20 times, the given code was wrong in **10**: 501 passes handed out, Asha getting 2, and once the set even lost an entry (501 people told yes, 500 recorded). The fix, with a lock per slot:

```
import java.util.*;
import java.util.concurrent.*;
import java.util.function.IntPredicate;

class Slot {
    final String name;
    final int capacity;
    private final Set<String> holders = new HashSet<>();       // guarded by this slot's lock

    Slot(String name, int capacity) { this.name = name; this.capacity = capacity; }

    // Both checks and the add are one step, per slot
    synchronized boolean book(String person) {
        if (holders.size() >= capacity || holders.contains(person)) return false;
        holders.add(person);
```

```
        return true;
    }

    synchronized int booked() { return holders.size(); }
}

class PassBooking {
    private final Map<String, Slot> slots = new ConcurrentHashMap<>();

    void addSlot(String name, int capacity) { slots.put(name, new Slot(name, capacity)); }

    boolean book(String slot, String person) {
        Slot s = slots.get(slot);
        if (s == null) throw new IllegalArgumentException("No slot " + slot);
        return s.book(person);                      // 9AM and 10AM never wait for each other
    }

    int booked(String slot) { return slots.get(slot).booked(); }
}

public class Darshan {
    public static void main(String[] args) throws Exception {
        PassBooking booking = new PassBooking();
        booking.addSlot("9AM", 500);
        booking.addSlot("10AM", 500);
        int got = rush(2000, i -> booking.book("9AM", "devotee" + i));
        System.out.println("2000 people, 500 passes: " + got + " got one, " + (2000 - got)
                + " refused");
        int same = rush(20, i -> booking.book("10AM", "Asha"));
        System.out.println("Asha taps 20 times: " + same + " pass");
        System.out.println("Booked: 9AM " + booking.booked("9AM") + ", 10AM " + booking.booked("10AM"));
    }

    // Given: runs task(0) .. task(n-1) on n threads that all start together; counts the trues.
    static int rush(int n, IntPredicate task) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(n);
        CountDownLatch go = new CountDownLatch(1);
        List<Future<Boolean>> results = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            int id = i;
            results.add(pool.submit(() -> { go.await(); return task.test(id); }));
        }
        go.countDown();
        int trues = 0;
        for (Future<Boolean> f : results) if (f.get()) trues++;
        pool.shutdown();
        return trues;
    }
}
```

```
2000 people, 500 passes: 500 got one, 1500 refused
Asha taps 20 times: 1 pass
Booked: 9AM 500, 10AM 1
```

Same output in 10 runs out of 10. The two checks and the add are one `synchronized` method on `Slot`, so slots never wait for each other.

**Part C.** (1) Optimistic: two people rarely edit the same profile at once; a `@Version` column catches the rare clash. (2) Pessimistic, or a conditional update (`WHERE status = 'FREE'`): thousands fight over each seat, so optimistic retries would pile up. (3) Optimistic, with a version check and a merge or "someone else changed this" message; never hold a lock while a person types. (4) Pessimistic, or the one-statement update `UPDATE wallet SET balance = balance - :amt WHERE id = :id AND balance >= :amt`: one hot row means constant collisions for optimistic retries.

## Exercise 9 — thinking out loud

| # | Better |
|---|---|
| 1 | "I need to know when two date ranges overlap. Let me work it out here... they clash when each starts before the other ends: `from1 < to2` and `from2 < to1`. Checking the edge: if one ends the day the other starts, that's false, which is what we want." |
| 2 | "Pricing has weekday, weekend and long-rental rules, and more will come each season, so I'll put it behind an interface. Each rule is a class, and booking code never changes." |
| 3 | "On one server, the per-car lock keeps bookings safe. With several servers I'd move that into the database as a conditional update, and split data by city, since a booking never spans cities." |
| 4 | "I'm indexing bookings by car, so checking whether a car is free only looks at that car's bookings, not every booking in the system." |
| 5 | "One lock would be correct and simpler. The cost is that booking a Swift waits behind booking an SUV, though they never conflict. A lock per car gives the same safety without that. With only a handful of cars, I'd happily use one lock." |

**Part B** (one way to open): "Let me check scope first. The users are delivery agents and customers; the flows are drop off, pick up with a code, and returning parcels nobody collected. Questions: are there several compartment sizes, and can a small parcel use a bigger one? How long do we keep a parcel? How are codes made, and do we limit wrong tries? One locker station or many? ... I'll assume three sizes, smallest-that-fits, three days, and a six-digit code from a generator I pass in, so tests can predict it. I'll design for the allocation rule changing. I'll start with the entities: Locker, Compartment, a Size enum, and Parcel as a record."

# Cheat sheet: Phase 4 on one page

| Topic | Remember |
|---|---|
| **1. Requirements** | 3–5 minutes, 6–8 grouped questions: actors, core flows, rules and numbers, edge cases, non-functional, scope cuts. Write R and NF lines and "out of scope". Every NF line must change something in the design. |
| **2. Entities** | Nouns → candidates, verbs → methods, then filter: entity (has an ID), attribute, value object (a record), enum (a fixed set differing in data), actor, out of scope. Put each method where its data is. |
| **3. UML** | `——>` association · `◇——` aggregation · `◆——` composition · `——▷` extends · `- - -▷` implements · `- - ->` uses briefly. Multiplicity at the far end. Entities, arrows, interfaces, service, repositories, in that order. |
| **4. Layers** | Driver parses and prints; service runs use cases; repository stores and finds; entity guards its own state. Arrows point one way. Same as Spring's controller, service, repository and entity. |
| **5. Repositories** | An interface in domain words, a `ConcurrentHashMap` behind it. `Optional` for one, a copy for many, no business rules, `save()` after every change. |
| **6. Testing** | Nothing created inside a service: repositories, outside systems, the `Clock` and strategies all come through the constructor. Fakes in tests; `main` or Spring picks the real ones. |
| **7. Exceptions** | Unchecked for domain errors; built-ins first ( `IllegalArgumentException` , `IllegalStateException` ); custom when the caller reacts differently. Expected outcomes are return values. Never swallow; keep the cause. |
| **8. Concurrency** | Find the check-then-act. Lock the smallest thing that covers it, or use one atomic call. Two locks: same order every time. Slow calls outside locks. Several servers: the database decides (conditional update). Optimistic when conflicts are rare, pessimistic when they're common. |
| **9. Out loud** | Say why, not what. Options, a choice and a reason. No silence over 30 seconds. Concede, then explain. |

## Mistake log

One row per mistake from the exercises, in your own words.

| Date | Section | What I did | What's right | Rule to remember |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

**How this connects to the rest.** Go back to one Phase 5 solution you've already done, say Hotel Booking, and look for each of the nine topics in it: the R and NF lines, the entity choices, the diagram, the layers, the repository (there, plain maps), what's passed in, the exceptions, the per-room lock. Then add the two things Phase 5 kept out to stay interview-sized: a repository interface and one JUnit test with a fake. After that, Phase 6 (mock interviews) is next.